Evidence-Driven Memory Retrieval for a Multilingual, Provider-Agnostic AI Client

Benchmarks, findings and product changes from the Turing AI project

Ömer Girgin · October 2026 · Code, data and full results: github.com/girginomer10/turing-ai, folder **benchmarks/**.

0.44 → 0.995	**22 · 9 · 6**	**+0.36**	**21–48%**
MRR@10 on personal memory, keyword search to the best pipeline (needs an embedding profile and a reranker; not the default setup)	embedding models (25 configurations) · rerankers · LLM judges	MRR for finding events again after the fading fix (all personas; 95% CI +0.26 to +0.45)	of unanswerable queries still got memory injected when an LLM judge decided at the prompt's 0.75 bar (six judges, wide CIs)

Summary

Turing AI is a native iOS, iPadOS and macOS app, with an Android port, where people bring their own provider keys and keep one local, portable memory across 50 model provider presets. Whether that memory helps depends on retrieval: finding the right earlier fact in Turkish or English among near-duplicates, and staying silent when nothing applies. I built two reproducible benchmarks around the app's real Swift retriever, evaluated 22 embedding models (25 configurations), 9 rerankers and 6 LLM judges with confidence intervals, and used the results to find and fix five defects in the product. Quality is reported as MRR@10: the average of 1/rank of the first correct memory among the top 10 (1 means always first, 0 never found). Where a configuration was selected, the selection used DEV personas and the score is reported on disjoint TEST personas; the headline figures pool all eight personas and are labelled as such.

The main findings: (1) models differ far more on personal memory than on encyclopedic text, and crossing languages is what separates them; (2) a second-stage reranker lifts a fully local pipeline from 0.887 to 0.964 and a hosted one to 0.995, the benchmark's ceiling (these numbers apply with Smart memory off; see section 4); (3) for saying “nothing here helps”, LLM judges inject less irrelevant memory than score thresholds at about 99% answer retention, although the judges differ and the intervals are wide; (4) the benchmarks exposed concrete bugs that no unit test had caught, such as memories becoming unfindable after a month, and a candidate pool that did not contain the answer for 19% of queries.

What a default user gets. No embedding profile exists until the user adds one, so out of the box the app ranks locally and Smart memory (on by default) judges the first 40 lexical matches, the pool that missed the answer for 19% of queries in the benchmark. The 0.964 and 0.995 pipelines need an embedding profile, a reranker (paid for the hosted ones) and Smart memory switched off; the numbers describe what is achievable, not the default.

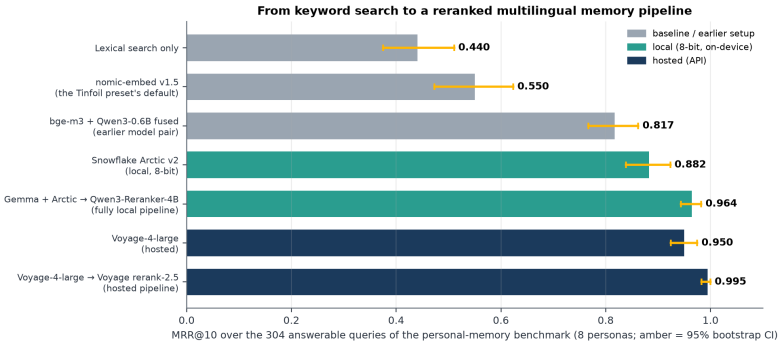


Figure 1. *MRR@10 for seven systems, from lexical search to a hosted two-stage pipeline (amber: 95% bootstrap CIs). The local pipeline (0.964) is not significantly different from Voyage-4-large alone (0.950; paired difference +0.015, CI −0.013 to +0.042).*

1 The problem and the system

People split their work across providers: a privacy-focused enclave for sensitive topics, ChatGPT and Claude side by side, free or anonymous models when cost matters. Memory, however, usually lives inside one provider's account. Turing AI keeps it on the device, in a local database, and injects only what a request needs into whichever model the user picks. Each embedding model keeps its own vector space and rankings are combined, so adding or changing a provider never forces re-embedding everything or migrating memory.

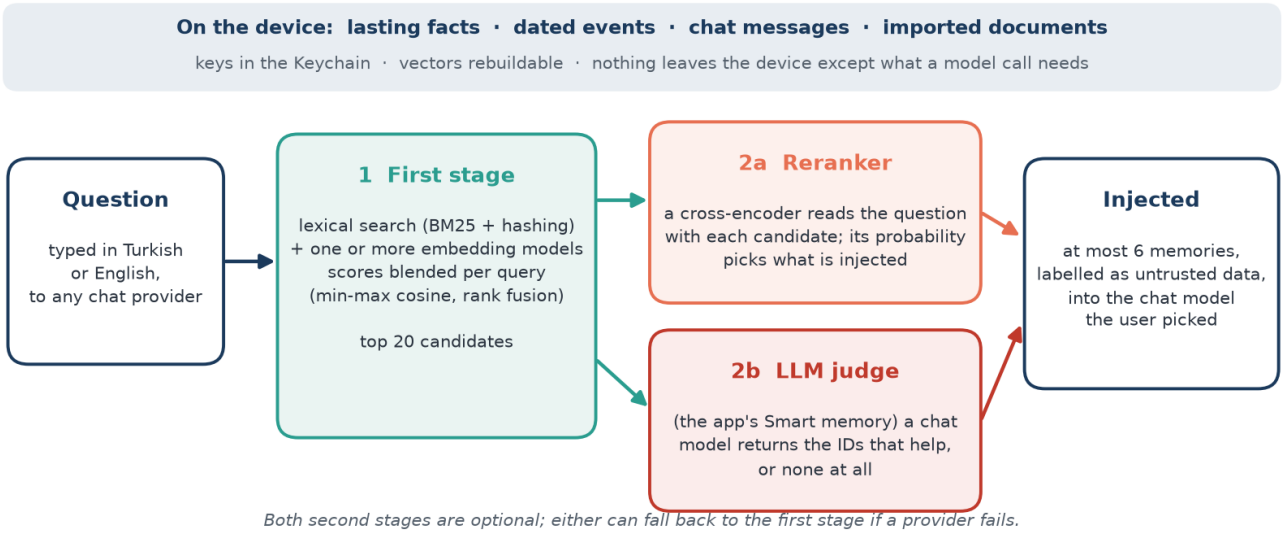


Figure 2. *The recall pipeline. A first stage blends lexical search with one or more embedding models and keeps up to 20 candidates (13 on average). Two optional second stages decide what is injected: a cross-encoder reranker, or an LLM judge (the app's "Smart memory" feature, on by default and taking precedence over the reranker). Either falls back to the first stage on failure.*

2 How I measured it

Retrieval quality was previously a matter of intuition, so I made it measurable. Everything below runs the **production retriever** (Swift, *HybridMemoryRetriever*), with embedding scores produced by local models through llama.cpp or by hosted APIs, fed in exactly as the app does.

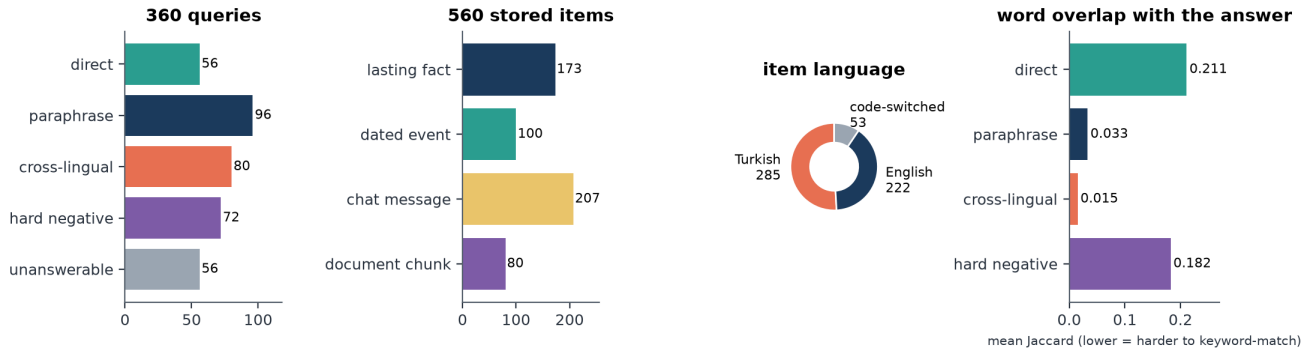


Figure 3. *The personal-memory benchmark: 8 invented personas, 560 stored items and 360 queries (each persona's store holds 67–73 items). Paraphrase and cross-lingual queries share almost no words with their answer (mean Jaccard 0.03 and 0.02), so keyword matching cannot solve them. Hard negatives have a near-duplicate distractor; unanswerable queries ask something plausible that no item answers.*

- **Two datasets.** I built the evaluation harness around XQuAD, a public set of 1,190 parallel Turkish/English questions over Wikipedia paragraphs (six monolingual, cross-lingual and mixed settings), and a synthetic personal-memory set that I created (above). MRR is computed over the 304 answerable queries; the 56 unanswerable ones measure false injection.
- **Label quality.** Four independent annotator agents re-labelled every query blind (no labels, no query types) and agreed with the authored labels on 360 of 360 queries (Cohen's κ = 1.0). This checks label validity, not realism, and each query had one annotator.

- **Statistical protocol.** Where something was selected (the fusion, the fade policy, the confidence bar, the XQuAD models) it was selected on DEV personas or paragraphs and reported on the disjoint TEST ones; the other figures, including the leaderboards and the reranker comparison, pool all eight personas, and the best reranker was picked after seeing them. DEV and TEST personas were written by the same process, so the split guards against selection bias, not against dataset bias. Intervals use a two-stage (persona, then query) or paragraph-cluster bootstrap. XQuAD comparisons also use a sign-flip permutation test; the personal-memory p-values come from the bootstrap. There is no multiplicity correction.
- **Self-audit.** I found and fixed my own harness bugs: a port collision that silently reused another model's server, random tie-breaking that made runs differ, a race in the spend ledger, and a metric that diluted effects with unanswerable queries. Python re-implementations of the score blend reproduce the Swift ranks in 42,840 of 42,840 cases.
- **Scope and cost.** 22 distinct embedding models in 25 configurations (8 local 8-bit, 17 hosted), 9 rerankers, 6 chat models running the app's own prompt. Every provider had a hard spend cap; the hosted study cost about $10 (details at the end).

Terms used in this report

MRR@10	Mean reciprocal rank: the average of 1/rank of the first correct memory, counting only the top 10 (1 = always first, 0 = never found).
Bootstrap 95% CI	A range obtained by re-sampling personas (then queries), or paragraphs for XQuAD, thousands of times. Overlapping intervals do not mean two systems are equal, and separate intervals are not required for a difference to be real: comparisons here use **paired differences** over the same queries.
Wilson interval	A 95% interval for a proportion such as “injected on 21% of unanswerable queries”, reliable for small counts.
8-bit GGUF	A compressed model file format run locally with llama.cpp; 8-bit weights use about a quarter of the memory of 32-bit ones.
Counterfactual aging, shield	Aging: every stored event is pretended to be 30, 90 or 365 days older to test fading. Shield: events already ranked in the top 3 keep their full weight.
DEV / TEST	Disjoint halves of the data: choices (a model, a fusion, a threshold) are made on DEV and scores are reported on TEST.
Embedding model	Maps text to a vector; related texts get close vectors, also across languages. **Cosine** measures closeness; **min-max cosine** rescales it per query to 0–1.
RRF	Reciprocal rank fusion: combines several rankings by summing 1/(60 + rank).
Reranker (cross-encoder)	A model that reads the question and one candidate together and scores the pair; slower but more accurate than comparing vectors.
Smart memory	The app's existing, default-on feature: a chat model reads the candidates and returns the IDs that help.
Unanswerable query	A plausible question that no stored item answers; the right behaviour is to inject nothing (“false injection” = injecting anyway).

3 Which embedding model?

Quality spreads widely on personal memory: from 0.55 (nomic-embed v1.5, still the default of one provider preset) to 0.95 (Voyage-4-large), while lexical search alone scores 0.44. On encyclopedic XQuAD the strong models sit within 0.016 of each other, so a benchmark that looks like the real data matters.

What the comparison shows

- **Parameter count alone does not decide.** Across families, a 300M-parameter EmbeddingGemma (0.871) matches Qwen3-Embedding-8B (0.875); within the Qwen3 family, larger models tended to score higher (embedding 0.770 → 0.845 → 0.875 for 0.6B, 4B, 8B, mixing local 8-bit and hosted builds; rerankers 0.862 → 0.970 → 0.980, with overlapping intervals above 0.6B). Voyage-4-large leads: its paired advantage is +0.026 [+0.001, +0.050] over the next best (Nemotron-8B with prefixes) and +0.027 [+0.006, +0.049] over Voyage-4, uncorrected for the many comparisons.
- **Local matched hosted where both were run.** The same model run locally in 8-bit and hosted at full precision scored the same for the two models tested (bge-m3 0.828 vs 0.828; EmbeddingGemma 0.871 vs 0.873), so quantization did not matter there.

- **Crossing languages separates the models** (Figure 5): nomic v1.5, Mistral embed and OpenAI text-embedding-3-small collapse when the question and the memory are in different languages (0.05, 0.19, 0.47), while Voyage-4-large keeps 0.925.
- **Input formats matter** (Figure 6): formatting text as each model card prescribes adds up to +0.07. The app was sending raw text for EmbeddingGemma, Arctic and E5 models; that is now fixed. The Nemotron format (+0.069 on the 8B model) is applied on OpenAI-compatible routes; NVIDIA NIM already receives it as *input_type*.

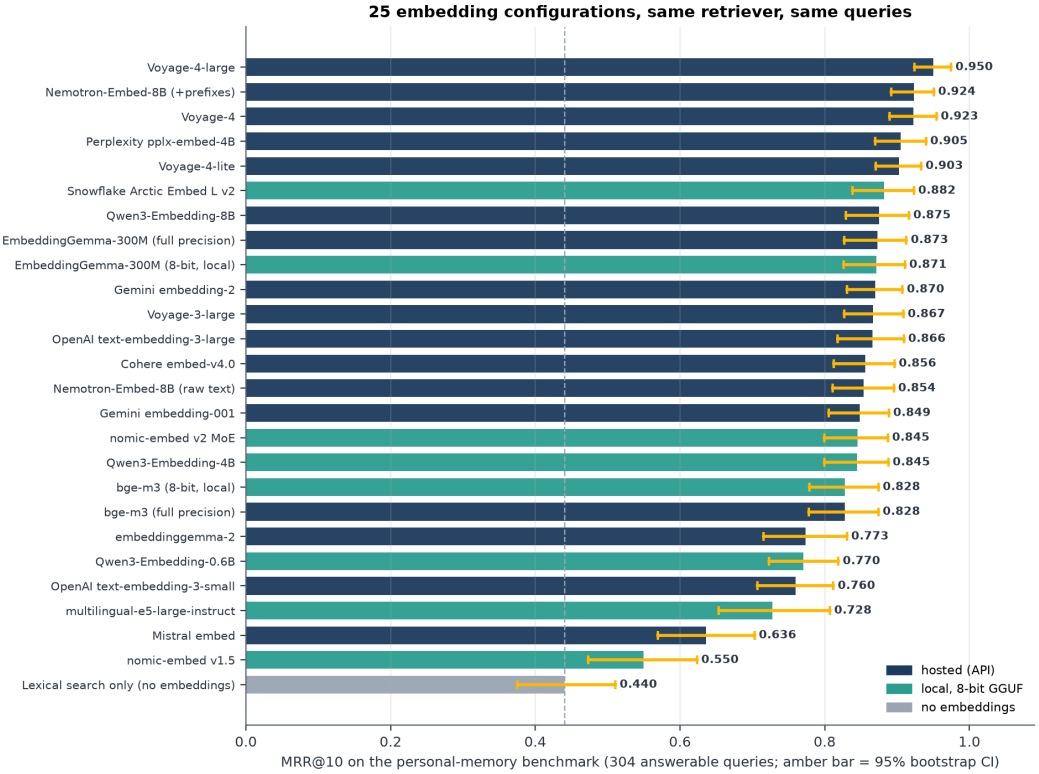


Figure 4. *All 25 embedding configurations (all eight personas) on the same retriever and queries, with 95% confidence intervals. Hosted models are navy, local 8-bit GGUF builds teal. The dashed line is lexical search alone.*

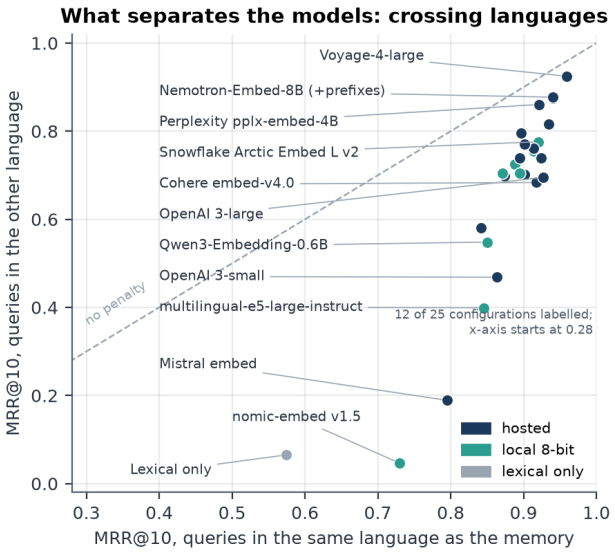


Figure 5. *Same-language vs other-language MRR@10. Points far below the diagonal pay a heavy price for crossing languages.*

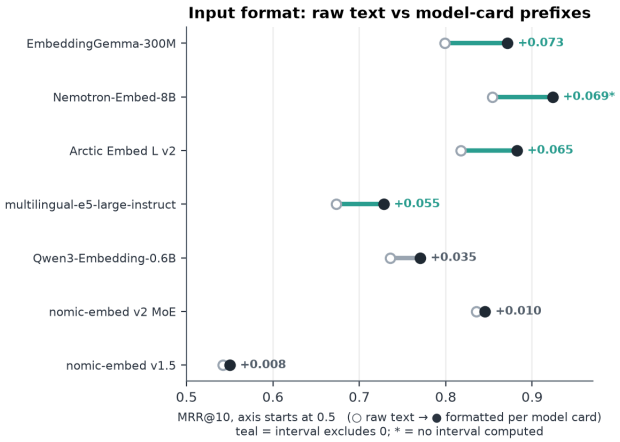


Figure 6. *Raw text (○) vs text formatted per model card (●). Gains: +0.073 EmbeddingGemma, +0.069 Nemotron-8B, +0.065 Arctic.*

Does combining models help?

Rank-fusing the two best local models (selected on DEV from 22 candidates) gives 0.867 against 0.841 for the best single model on DEV (EmbeddingGemma) evaluated on TEST (+0.026, 95% CI −0.003 to +0.060, p = 0.08): small and not significant. Arctic alone scores 0.876 on TEST, above the fusion, and an equal-weight member that is weak hurts (−0.102 for nomic v1.5). The data therefore do not show that fusion raises accuracy; the app's multi-profile design is justified by not forcing re-embedding when a provider changes, not by a measured accuracy gain.

4 A second stage: rerankers

A cross-encoder reads the question together with each of the first stage's top 20 candidates and its probability decides what is injected. The answer is inside the first stage's candidates (up to 20, 13 on average with Voyage-4-large because of a relevance gate and an event limit) for 99.7% of queries with the local models and 100% with Voyage-4-large, so a reranker is almost never starved.

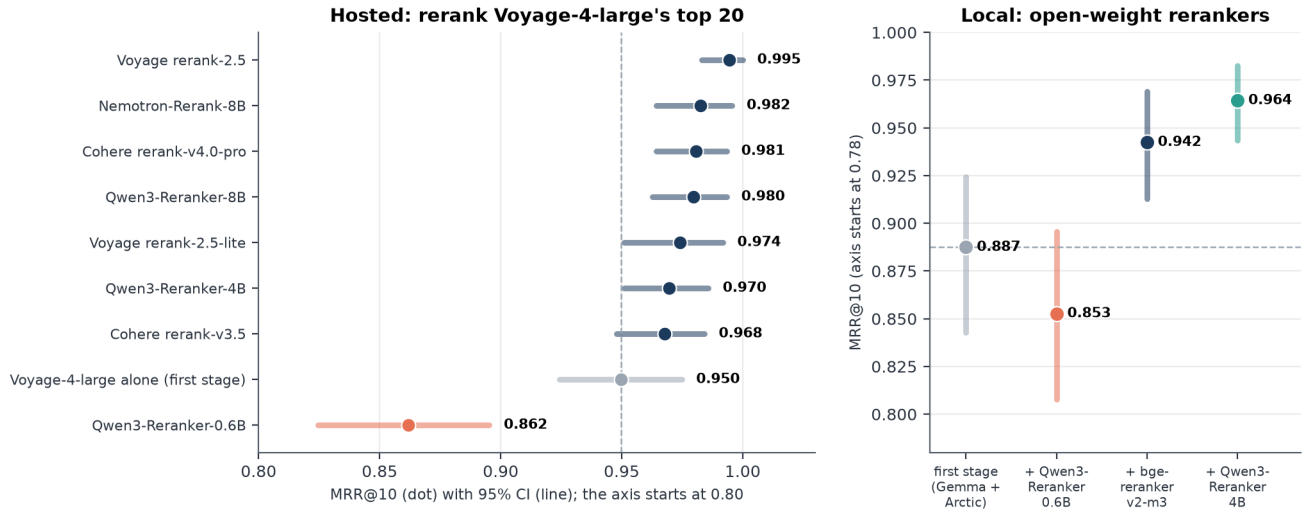


Figure 7. *Dots with 95% CIs (axes start at 0.80 and 0.78). Left: eight hosted rerankers on top of Voyage-4-large (0.950). Voyage rerank-2.5 reaches 0.995, the benchmark's ceiling. A too-small reranker (Qwen3-0.6B, orange, 0.862) is worse than none. Right: open-weight rerankers on a fully local first stage; Qwen3-Reranker-4B lifts it from 0.887 to 0.964 and cross-lingual MRR from 0.78 to 0.96. The best entries were chosen after seeing these results, so they are optimistic.*

- **Where it helps.** Gains are largest when the first stage is weak (+0.135 on a bge-m3 + Qwen3-0.6B first stage) and small when it is already strong; the best hosted pipeline saturates the benchmark.
- **How scores must be used.** Feeding reranker scores through the first-stage blend (0.42 lexical weight) wasted them (0.907 vs 0.964): lexical noise diluted a much stronger judge, so selection now follows the reranker.
- **Precedence in the app.** Smart memory is on by default and takes precedence over the reranker, so the 0.964 and 0.995 pipelines apply when Smart memory is off and the reranker is on.
- **Cost and speed.** bge-reranker-v2-m3 takes about 0.2 s per query on a laptop; Qwen3-Reranker-4B about 4 s; hosted rerankers 0.3–0.9 s per query over the network.

5 Staying silent when nothing helps

Real memory systems fail quietly by injecting something irrelevant. 56 of the 360 queries are unanswerable. The app's **Smart memory** (on by default) already lets a chat model read the candidates and return the IDs that help, so the question is how well it stays silent and which candidates it should see. Before this work it judged the first 40 items of a lexical ranking, which injected on 11–41% of unanswerable queries but held the answer for only 81% of queries. Judging the embedding-ranked candidates (up to 20, 13 on average, against always 40) holds the answer for 100% (with Voyage-4-large) but, for four of six judges, raised the false-injection rate (the table's last two columns); pool size and ranking are confounded here. I ran Smart memory with the app's prompt, request format, budget and decoding rules on six chat models, and compared it with thresholds on a reranker.

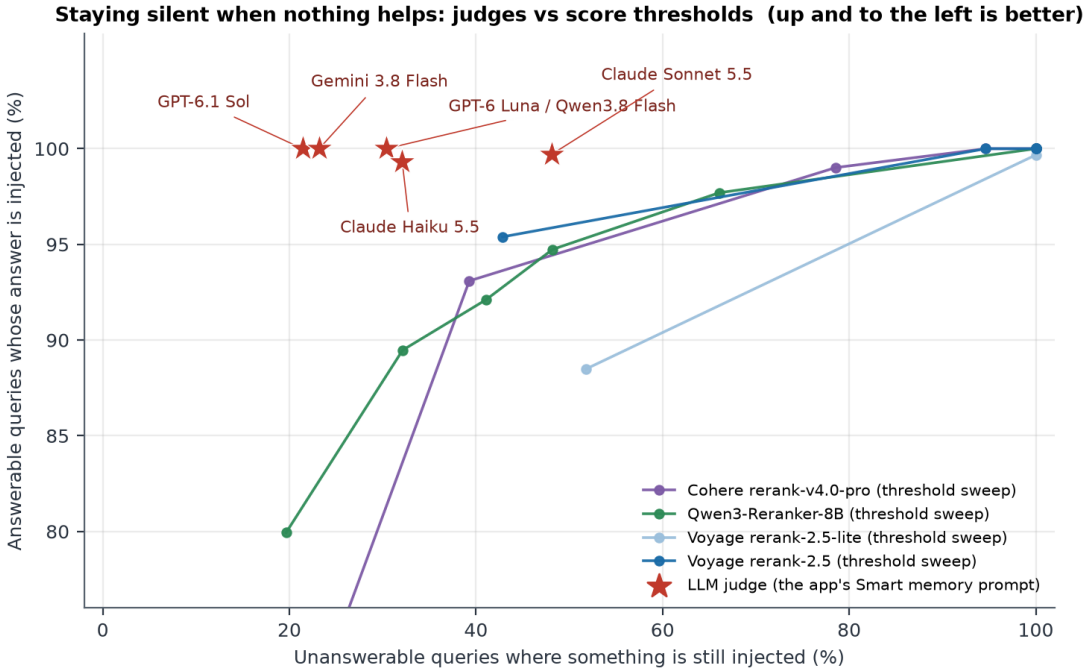


Figure 8. *Each curve is a reranker swept over thresholds: to inject less on unanswerable queries (move left) it must also drop correct answers (move down; the y-axis starts at 76%). Each star is an LLM judge on the embedding pool: all six keep 99–100% of answers while injecting on 21–48% of unanswerable queries. The reranker curves stop at the thresholds measured (τ up to 0.5 for rerank-2.5), so the comparison holds near 99% answer retention, not along the whole frontier.*

Judge (one run each)	Valid first answers (strict → tolerant)	Answer selected	Unanswerable queries injected (embedding pool) [95% CI]	Old lexical pool	Cost / 1,000 queries	Seconds / query
GPT-6.1 Sol	100% → 100%	1.000	21% [13%, 34%]	23%	$2.44	2.2
Gemini 3.8 Flash	100% → 100%	1.000	23% [14%, 36%]	11%	$1.44	4.4
Qwen3.8 Flash	97% → 100%	1.000	30% [20%, 43%]	24%	$0.19	2.5
GPT-6 Luna	100% → 100%	1.000	30% [20%, 43%]	38%	$0.14	2.1
Claude Haiku 5.5	100% → 100%	0.993	32% [21%, 45%]	18%	$0.25	1.7
Claude Sonnet 5.5	79% → 98%	0.997	48% [35%, 61%]	41%	$3.80	1.7

Intervals are Wilson 95% intervals over 56 unanswerable queries (54 for Sonnet on the embedding pool; on the old pool 37 for Sonnet after payment errors and 55 for Qwen) and overlap heavily: for example GPT-6 Luna (30%) and GPT-6.1 Sol (21%) cannot be told apart. Validity counts first attempts only and excludes provider errors; one Sonnet run on the old pool lost 84 calls to payment errors. For comparison, Qwen3-Reranker-8B at a threshold of 0.1 keeps 97.7% of answers but still injects on 66% of unanswerable queries. The harness is not fully app-faithful: output-token caps were 3,000 (Gemini) and 4,000 (GPT) against the app's 800, there was no 12 s timeout, temperature was 0 for Claude and Qwen (the app sends none), Qwen's reasoning was off and Gemini's low, Gemini, Claude and Qwen went through OpenRouter, and the decoder is a Python reimplementation. In the app a slow or truncated answer falls back to ordinary ranking, which injects memory: 39 of Gemini's 360 calls took over 12 s (4 on unanswerable queries), so its false injection would be at least 30% (17/56), not 23%. Rejected answers are likewise left out of the denominators here. Each judge ran once.

A higher selection bar. I re-analysed the stored judge answers (no new API calls, all six judges, rejected answers re-read with the tolerant decoder) with a stricter confidence bar than the prompt's 0.75. A higher bar can only remove selections, so false injection cannot rise by construction; the question is the trade. The rule was to take the highest bar at which no judge loses more than 2 points of answers selected on DEV personas (written down in the analysis script after 0.85 had been picked, and it reproduces 0.85). On TEST personas, pooled over the six judges, anything injected on unanswerable queries fell from 41% to 23% of (judge, query) pairs (68/167 to 39/167) while answers selected went from 0.999 to 0.994 (906/907 to 902/907). By judge (TEST, 28 unanswerable queries each): GPT-6 Luna 39% → 36%, Claude Haiku 5.5 43% → 0%, Claude Sonnet 5.5 59% → 7%, Qwen3.8 Flash 46% → 39%; GPT-6.1 Sol and Gemini unchanged. The app now ships 0.85. Counts are small and each judge ran once, so the direction is dependable and the size is not; the table above was measured at 0.75.

6 What the benchmarks changed in the product

The point of measuring was to fix things. Five defects surfaced and one tuning change followed; each was made on the evidence of these benchmarks, on Swift and, where applicable, Android, and the Swift changes are exercised by the automated tests (the confidence bar has no dedicated test).

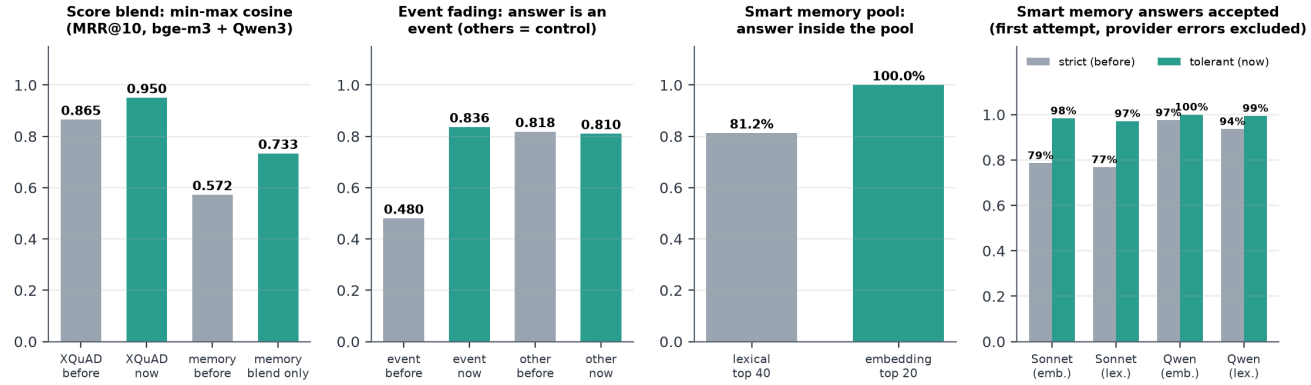


Figure 9. *Before and after for four of the fixes. Blend: XQuAD 0.865 → 0.950; personal memory 0.572 → 0.733 with the blend change alone (+0.160, 95% CI +0.123 to +0.201; the shipped retriever with all changes scores 0.817). Events: the answer-is-an-event MRR rises from 0.48 to 0.84 at a cost of −0.008 on other queries over all personas (−0.013, CI −0.036 to −0.001, on TEST). Smart memory: the pool contains the answer for all queries with Voyage-4-large (99.7% with local models), and tolerant decoding accepts 97–100% of first-attempt answers.*

Finding	Evidence	Change in the app
The semantic score was a rank percentile, almost flat at the top, so lexical noise decided the order	XQuAD 0.865 → 0.950; personal memory 0.572 → 0.733 (+0.160)	Per-query min-max cosine in the blend
Events halved in weight every 14 days: a month-old event was nearly unfindable	Answer-is-event MRR 0.48 → 0.84 (+0.356, CI +0.262 to +0.453); 17 policies compared by counterfactual aging	45-day half-life, 0.4 floor, shield for top-ranked events
Models need their documented query/document prefixes; the app sent raw text	+0.073 EmbeddingGemma, +0.065 Arctic (Nemotron-8B +0.069)	Correct formats for Gemma, Arctic, E5 and Nemotron (except on NIM); stored vectors are rebuilt
Smart memory judged only the first 40 of a lexical ranking, which held the answer for 81% of queries	Embedding top 20 held it for 100%; judges selected it 99–100% of the time, but four of six injected more on unanswerable queries	With an embedding profile (iOS/macOS), the judge sees the embedding-ranked top 20
Its strict JSON decoder discarded whole correct answers: Claude appended “Wait…” corrections, Qwen returned a bare array, GPT mistyped a UUID	First-attempt validity, strict → tolerant: Claude Sonnet 5.5 78.6% → 98.3%; Qwen 97.5% → 100% (embedding pool)	Tolerant decoder, still rejecting answers made only of invented IDs
Smart memory kept every entry the model rated at least 0.75, so judges still injected on 21–48% of unanswerable queries	Rule set on DEV, reported on TEST, six judges pooled: injection on unanswerable queries 41% → 23%, answers selected 0.999 → 0.994 (28 unanswerable queries per judge)	Selection bar raised to 0.85 in the decoder; prompt unchanged

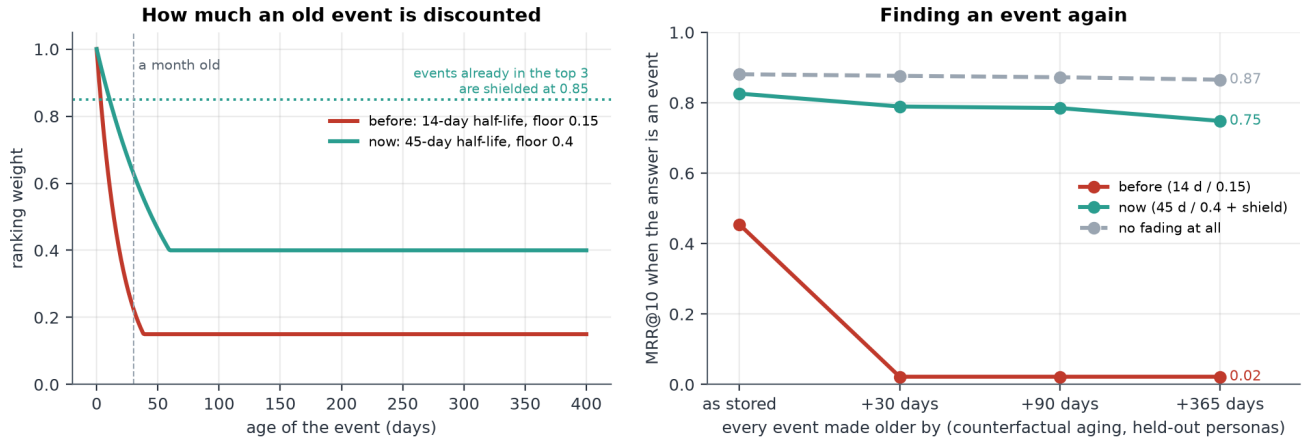


Figure 10. *The event-fading fix in detail. Left: the old policy discounted a month-old event to 0.23 of its weight. Right: on held-out personas with every event artificially aged, the old policy lost the event as soon as it was a month old (0.45 → 0.02); the new one keeps it findable (0.75 even after a year, against 0.87 with no fading, a gap of 0.117). Most of that older-event gain comes from the shield: the old half-life with the shield scores the same 0.789, 0.785 and 0.748 at +30, +90 and +365 days; the longer half-life and higher floor mainly help events as stored (0.826 vs 0.793).*

Besides the fixes I added an optional reranker stage (off by default; Voyage, Cohere, Jina, DeepInfra and self-hosted */rerank* servers) with graceful fallback and usage logging, changed Ollama's default embedding model to EmbeddingGemma, and added a DeepInfra rerank client. Suggested reranker models follow the measurements (Voyage rerank-2.5, Cohere rerank-v4.0-pro, DeepInfra Qwen3-Reranker-8B); Jina's suggestion was not measured. 617 Swift tests pass, and CI builds macOS, iOS and Android.

Reproducibility and cost

Dataset files, harness, analysis scripts and every RESULTS.md are in **benchmarks/**. The raw intermediate outputs (*work/*) are git-ignored, so this report's figures can be rebuilt only after re-running the pipeline. The hosted experiments cost about $10 according to provider reports and list prices (OpenRouter ≈ $7.3, OpenAI ≈ $3, DeepInfra ≈ $0.1, free tiers for Voyage and Cohere); the local spend ledger records $8.97 and has no DeepInfra entry. Each provider had a spend cap. Keys live in a git-ignored file and are never printed or logged.